

Steepest-edge simplex algorithms for linear programming

John J. Forrest

IBM Research Division, T.J. Watson Research Center, Yorktown Heights, NY, USA

Donald Goldfarb*

IEOR Department, Columbia University, New York, USA

Received 12 September 1991

Revised manuscript received 3 August 1992

We present several new steepest-edge simplex algorithms for solving linear programming problems, including variants of both the primal and the dual simplex method. These algorithms differ depending upon the space in which the problem is viewed as residing, and include variants in which this space varies dynamically. We present computational results comparing steepest-edge simplex algorithms and approximate versions of them against simplex algorithms that use standard pivoting rules on truly large-scale real-world linear programs with as many as tens of thousands of rows and columns. These results demonstrate unambiguously the superiority of steepest-edge pivot selection criteria to other pivot selection criteria in the simplex method.

Key words: Steepest-edge simplex method, large-scale linear programming, Devex variants.

1. Introduction

We consider steepest-edge simplex algorithms for solving linear programming problems of the form

$$\begin{aligned} &\text{minimize} && c^T x \\ &\text{subject to} && Ax = b, \\ &&& 0 \leq x \leq u, \end{aligned} \tag{1.1}$$

where A is an $m \times n$ matrix of rank m and $m < n$. Assuming nondegeneracy all variants of the primal simplex method proceed from vertex to adjacent vertex of the polyhedron of feasible solutions of (1.1) along edges that are “downhill” — i.e., along which the objective function decreases until further improvement is not possible. In the steepest-edge variant the edge chosen at each iteration is one that is “most

Correspondence to: Prof. Donald Goldfarb, Department of Industrial Engineering and Operations Research, Columbia University, New York, NY 10027-6699, USA.

* The research of this author was supported in part by NSF Grants DMS 85-12277, DMS 91-0619 and CDR 84-21402.

downhill” — i.e., steepest with respect to the objective function. Equivalently, the edge chosen is one that makes the largest (most obtuse) angle with the gradient c of the objective function.

It has been known since the early experiments of Kuhn and Quandt (1963) and Wolfe and Cutler (1963) that, in general, the steepest-edge simplex algorithm takes fewer iterations than other variants of the primal simplex method. However, because a straightforward implementation of the steepest-edge algorithm requires an excessive amount of time for each iteration, the algorithm was initially rejected as impractical.

The first practical steepest-edge type simplex algorithm was devised by Harris (1973), who incorporated it into a code named Devex. (Devex comes from the Latin word *devexus* for steep.) Numerical test results presented in (Harris, 1973) showed that the Devex code outperformed, by substantial margins both in terms of time and iteration counts, standard simplex algorithms which used partial pricing.

Harris’ algorithm chooses edges that are only approximately steepest, and it was not until the publication of (Goldfarb and Reid, 1977) that the first practical (exact) steepest-edge algorithm was described. The key to this algorithm was the development of formulas for updating the squares of the norms of the edge directions at a basic feasible solution (vertex) when a simplex pivot is performed. Goldfarb and Reid (1977) also reported numerical test results showing that their algorithm was somewhat superior to Harris’ algorithm and sometimes far superior to an algorithm which used the so-called Dantzig most negative reduced cost column (edge) selection rule. Independently, Greenberg and Kalan (1975) developed recurrence formulas very similar to those given in (Goldfarb and Reid, 1977) for updating weights equal to the squares of the norms of the edge directions minus one.

Our purpose in writing this paper is twofold. First, it is to introduce several new steepest-edge algorithms, including variants of both the primal and the dual simplex method. Three dual algorithms are developed that differ depending upon the chosen formulation of the dual problem. Also “projected” variants of both primal and dual steepest-edge algorithms are presented. These can be viewed as “pure” steepest-edge algorithms applied to the primal or the dual problem, as the case may be, after first eliminating certain variables. Dynamic versions of these algorithms, in which the space onto which the projection is made is allowed to vary, are also discussed. Our second purpose is to present computational results which demonstrate the efficiency of using primal and dual and both exact and inexact steepest-edge simplex algorithms. Such tests are relevant at this time because advances in computer hardware and software technology have affected the relative speed of doing different operations in simplex codes and have made it possible to solve much larger problems in a reasonable amount of time than was previously possible. Consequently, the advantage of using a steepest-edge approach has dramatically increased.

The paper is organized as follows. We first review the steepest-edge simplex algorithm for standard form linear programs and discuss how this algorithm adapts to problems with upper bounds on the variables. We then present in Section 3, three dual steepest-edge simplex algorithms and recurrence relations that make these algorithms

practical. In Section 4 we describe inexact variants of both primal and dual steepest-edge algorithms, and in Section 5 we present several “projected” variants including ones in which the space onto which the projection is made varies dynamically. Finally, in Section 6 the results of extensive computational tests comparing the use of various exact and inexact steepest-edge pivot rules in the OSL linear programming code are reported. These results are for most of the larger and more difficult of the problems in the Netlib test set (Gay, 1985) and some additional real-world problems with tens of thousands of rows and columns.

2. Primal algorithms

In this section we review the primal steepest-edge simplex algorithm proposed in (Goldfarb and Reid, 1977) for solving the standard form linear programming problem:

$$\begin{aligned} &\text{minimize} && c^T x \\ &\text{subject to} && Ax = b, \\ &&& x \geq 0, \end{aligned} \tag{2.1}$$

where A is an $m \times n$ matrix of rank m and $m < n$.

Consider a single step of the simplex method applied to (2.1) and let B and N denote the submatrices of A corresponding to basic and nonbasic columns, respectively. To simplify our exposition, we will henceforth assume that the first m columns of A and components of x are basic at the start of the step. This step is of the form

$$\bar{x} = x + \theta \eta_q,$$

where

$$x = \begin{bmatrix} B^{-1}b \\ 0 \end{bmatrix} \tag{2.2}$$

is the basic feasible solution at the start of the step, η_q is one of the set of edge directions

$$\eta_j = \begin{bmatrix} -B^{-1}N \\ I \end{bmatrix} e_{j-m}, \quad j = m+1, \dots, n, \tag{2.3}$$

emanating from the vertex x , and θ is the length of the step. e_i denotes the i th column of the identity matrix I . The edge direction η_q must be “downhill” — i.e., η_q must make an obtuse angle with the gradient c of the objective function, or equivalently, the reduced cost $\bar{c}_q \equiv c^T \eta_q$ must be negative. Notice that some of the edge directions (2.3) may be infeasible if x in (2.2) is degenerate.

In the steepest-edge simplex algorithm, the edge η_q that is chosen, or equivalently, the nonbasic variable x_q that is chosen to enter the basis, is the edge that is most downhill relative to the gradient c ; i.e.,

$$\frac{c^T \eta_q}{\|\eta_q\|} = \min_{j > m} \left\{ \frac{c^T \eta_j}{\|\eta_j\|} \right\}$$

where $\|\cdot\|$ denotes the Euclidean norm.

To implement this pivot rule in a practical manner, Goldfarb and Reid (1977) derived recurrence formulas for updating the quantities $\gamma_j = \|\eta_j\|^2$, for all nonbasic indices j , after each pivot. If variable x_q replaces variable x_p in the basis, then it is easy to derive the edge direction recurrence formulas

$$\bar{\eta}_p = -\eta_q / \alpha_q, \quad (2.4a)$$

$$\bar{\eta}_j = \eta_j - \eta_q \bar{\alpha}_j, \quad j > m, j \neq q, \quad (2.4b)$$

and from these the recurrence formulas

$$\bar{\gamma}_p = \gamma_q / \alpha_q^2, \quad (2.5a)$$

$$\bar{\gamma}_j = \gamma_j - 2\bar{\alpha}_j \alpha_j^T v + \bar{\alpha}_j^2 \gamma_q, \quad j > m, j \neq q, \quad (2.5b)$$

where $\alpha_j = \sigma^T a_j$, $\bar{\alpha}_j = \alpha_j / \alpha_q$, $Bw = a_q$, $B^T v = w$, and $B^T \sigma = e_p$. Note that $\alpha_q = w_p$, and that α_j and $\bar{\alpha}_j$ are the components of the p th rows of $B^{-1}A$ and $\bar{B}^{-1}A$, respectively, where $\bar{B}$ is the basis matrix after the pivot. Premultiplying both sides of (2.4a) and (2.4b) by c^T , we see that the numbers $\bar{\alpha}_j$ are also needed if the ‘‘reduced costs’’ are updated. Also γ_q , and hence $\bar{\gamma}_p$, can be computed explicitly, since $\gamma_q = 1 + w^T w$.

Updating the γ_j requires some extra computation: the solution of an additional equation $B^T v = w$ and the inner products of v with nonbasic columns a_j corresponding to $\bar{\alpha}_j$ that are nonzero. Fortunately, in sparse problems, most of the $\bar{\alpha}_j$ are zero. On the basis of the computational experience reported in Section 5, the increase in the computational time per step over the standard simplex algorithm is roughly 8%. This is a lot smaller than the increase reported in (Goldfarb and Reid, 1977) and is due to the advantage in pricing and in solving the two systems of equations with the same matrix B^T that can be taken of the architecture of the IBM RISC System/6000 used in our tests. Extra storage for the γ_j 's is, of course, also needed. Time must also be spent computing the γ_j 's initially.

If a variable x_j has an upper bound u_j on its value in addition to having a lower bound of zero (i.e., in problem (2.1) $x_j \geq 0$ is replaced by $0 \leq x_j \leq u_j$) then this variable can be nonbasic at either its lower bound of zero or at its upper bound. It is obvious that when the variable x_j chosen to enter the basis corresponds to the latter case (i.e., when x_j is at its upper bound and is decreased) the edge direction is the negative of what it is in the former case (i.e., when x_j is equal to zero and is increased). Consequently, the quantities γ_j and the recurrence formulas for them do not change.

3. Dual algorithms

In this section we present three steepest-edge variants of the dual simplex method and derive formulas for updating the squares of the norms of the edge directions in the dual problem that enable these variants to be implemented efficiently. To simplify matters we will first derive algorithms for the standard form problem (2.1). Then we will extend our results to problems with upper bounds on the variables.

3.1. Dual algorithm 1

Consider a single step of the dual simplex method applied to problem (2.1), given the basic dual feasible solution (2.2). If we write the dual of (2.1) as

$$\begin{aligned} &\text{maximize} && b^T y \\ &\text{subject to} && A^T y \leq c, \end{aligned} \quad (3.1)$$

then the dual solution y corresponding to (2.2) satisfies (by complementarity)

$$B^T y = c_B, \quad (3.2a)$$

$$N^T y \leq c_N, \quad (3.2b)$$

where $c^T = (c_B^T, c_N^T)$ is partitioned into basic and nonbasic parts conformally with $A = [B, N]$.

On a dual simplex step, choosing variable x_p to leave the basis corresponds to relaxing the p th equation in (3.2a); i.e., the requirement that the p th constraint $a_p^T y \leq c_p$ be satisfied as an equality is dropped. Note that we are using a_p to denote the p th column of A . Since dual solutions of the form $y(\theta) \equiv y + \theta \rho_p$, where $B^T y = c_B$, $B^T \rho_p = -e_p$ and $\theta \geq 0$ satisfy $a_i^T y(\theta) = c_i$, $1 \leq i \leq m$, $i \neq p$ and $a_p^T y(\theta) \leq c_p$, the rows of $-B^{-1}(\rho_i, i = 1, \dots, m)$ are clearly the edge directions in the dual polyhedron $\{y \in \mathbb{R}^m \mid a^T \leq c\}$ emanating from the vertex $y = B^{-T} c_B$. In the dual simplex method, the edge direction ρ_p (or equivalently the leaving variable x_p) chosen must make an acute angle with the gradient b of the dual objective function; that is, $\rho_p^T b = -e_p^T B^{-1} b$ must be positive (x_p must be negative). If the edge ρ_p chosen is to be a steepest-edge, then we must have

$$\frac{b^T \rho_p}{\|\rho_p\|} = \min_{i \leq m} \left\{ \frac{b^T \rho_i}{\|\rho_i\|} \right\}.$$

As in the primal case, the practicality of using a steepest-edge pivot rule in the dual simplex method depends upon updating the squares of the edge direction norms $\beta_i = \rho_i^T \rho_i = \|\rho_i\|^2$, $i = 1, \dots, m$. If variable x_q replaces variable x_p in the basis, then

$$\bar{B}^{-1} = B^{-1} - \frac{w^- e_p}{w_p} e_p^T B^{-1}$$

where $w = B^{-1}a_q$. It then follows that

$$\bar{\rho}_p = (1/w_p)\rho_p, \quad (3.3a)$$

$$\bar{\rho}_i = \rho_i - (w_i/w_p)\rho_p, \quad i = 1, \dots, m, \quad i \neq p, \quad (3.3b)$$

and

$$\bar{\beta}_p = (1/w_p)^2\beta_p, \quad (3.4a)$$

$$\bar{\beta}_i = \beta_i - 2(w_i/w_p)\tau_i + (w_i/w_p)^2\beta_p, \quad i = 1, \dots, m, \quad i \neq p, \quad (3.4b)$$

where $B^T\sigma = e_p$ and $B\tau = \sigma$, and β_p can be computed explicitly as $\beta_p = \sigma^T\sigma$.

In general it should be less expensive to implement the recurrence formulas for the β_i than those for the γ_j . Both formulas require the solution of one extra equation ($B\tau = \sigma$ for (3.4b) and $B^T v = w$ for (2.5b)), but extra inner products are only required in the primal case ($a_j^T v$ for all j such that $\bar{a}_j \neq 0$). The latter computation can be rather expensive if $n \gg m$ and row p of $B^{-1}A$, the pivot row of the tableau, is not very sparse. Also note that the quantities $1/w_p$ and $-w_i/w_p$ in (3.3) and (3.4) are components of the "eta" vector used to update the factors of B . Moreover, computing β_i , $i = 1, \dots, m$, initially is usually less expensive than computing γ_j , $j = m+1, \dots, n$, since n is often greater than $2m$, and greater advantage of sparsity can be taken in computing $B^{-T}e_i$ than in computing $B^{-1}a_j$.

3.2. Dual algorithm 2

An alternative steepest-edge row selection strategy for the dual simplex method is obtained if one expresses the dual problem (3.1) in terms of nonnegative dual slack variables s as well as the free dual variables y . Specifically, if we write (3.1) as

$$\begin{aligned} &\text{maximize} && b^T y \\ &\text{subject to} && A^T y + s = c, \\ &&& s \geq 0, \end{aligned} \quad (3.5)$$

then the dual solution (y, s) corresponding to the basic dual feasible solution (2.2) satisfies the $(n+m) \times (n+m)$ system of equations

$$\begin{bmatrix} B^T & 0 & I \\ N^T & I & 0 \\ 0 & 0 & I \end{bmatrix} \begin{pmatrix} y \\ s_N \\ s_B \end{pmatrix} = \begin{pmatrix} c_B \\ c_N \\ 0 \end{pmatrix} \quad (3.6)$$

and the requirement that $s_N \geq 0$. (s is partitioned into basic and nonbasic parts in the same manner as c .) It is easy to demonstrate that the edge directions of the polyhedron $\{(y, s) \in \mathbb{R}^{m+n} \mid A^T y + s = c, s \geq 0\}$ emanating from the vertex $(y, s) = (B^{-T}c_B, c_N - N^T B^{-T}c_B)$ are the last m columns of the matrix

$$\begin{bmatrix} B^{-T} & 0 & -B^{-T} \\ -N^T B^{-T} & I & N^T B^{-T} \\ 0 & 0 & I \end{bmatrix},$$

which is the inverse of the matrix in (3.6). If we let $\rho_i = e_i^T [-B^{-1}, B^{-1}N, I]$ for $i = 1, \dots, m$, then the recurrence formulas (3.3a) and (3.3b) still apply to these edge directions. The recurrence formulas (3.4a) and (3.4b) for $\beta_i = \|\rho_i\|^2$, $i = 1, \dots, m$, also still apply with the change that

$$B\tau = \sigma + v \quad (3.7)$$

where $v = \sum_{j=m+1}^n a_j a_j$ and $\alpha_j = \sigma^T a_j$, the j th element of the pivot row. Also, β_p can be computed explicitly as $\beta_p = \sigma^T \sigma + a^T a$. Note that the vector τ used in (3.4b) when formulation (3.1) of the dual problem is used corresponds to τ in (3.7) with $v = 0$.

3.3. Dual algorithm 3

A third dual steepest-edge simplex algorithm can be obtained by eliminating the free dual variables y from (3.5). This can be done by choosing an $(n-m) \times n$ matrix F so that the $n \times n$ matrix $G^T \equiv [A^T, F^T]$ is nonsingular, and premultiplying the equality constraints in (3.5) by $G^{-T} \equiv [Y^T, Z^T]$ where Y and Z have m and $n-m$ columns, respectively. This yields the equivalent system of linear equations

$$y + Y^T s = Y^T c,$$

$$Z^T s = Z^T c.$$

Hence (3.5) can be rewritten as

$$\begin{aligned} &\text{maximize} && b^T Y^T (c - s) \\ &\text{subject to} && Z^T s = Z^T c, \\ &&& s \geq 0. \end{aligned} \quad (3.8)$$

Note that the columns of Z form a basis for the null space of A .

Also note that there are many possible choices for F (and Y and Z). For example if $A = [B, N]$ and B is nonsingular, then we can choose $F = [0, I]$. We can also choose $F = Q_2$, where $A^T = [Q_1, Q_2] \begin{bmatrix} R \\ 0 \end{bmatrix}$ is the QR factorization of A^T . This gives $[Y, Z] = [Q_1 R^{-T} Q_2]$. For all allowable choices of F , Z_N is nonsingular, as we show below, where $Z = (Z_B, Z_N)$ is partitioned conformally with $A = [B, N]$.

We shall now show that the particular choice of F does not affect the edge directions of the feasible polyhedron of problem (3.8). Specifically, we shall show that given a basis B and dual basic feasible solution of problem (3.8),

$$s = \begin{pmatrix} s_B \\ s_N \end{pmatrix} = \begin{pmatrix} 0 \\ Z_N^{-T} Z^T c \end{pmatrix}, \quad (3.9)$$

corresponding to the primal solution (2.2), the edge directions emanating from (3.9) are the rows of the current primal simplex tableau $[I, B^{-1}N]$. With this goal in mind let

$$\hat{G} \equiv \begin{bmatrix} B & N \\ 0 & I \end{bmatrix}$$

and

$$[\hat{Y}, \hat{Z}] \equiv \hat{G}^{-1} = \begin{bmatrix} B^{-1} & -B^{-1}N \\ 0 & I \end{bmatrix}. \quad (3.10)$$

Since $A[Y, Z] = [I, 0]$,

$$\begin{bmatrix} B & N \\ 0 & I \end{bmatrix} [Y, Z] = \begin{pmatrix} I & 0 \\ S & T \end{pmatrix},$$

where $[S, T] = [0, I][Y, Z] = [Y_N, Z_N]$, and $Z_N = T$ is nonsingular. Consequently

$$[Y, Z] = [\hat{Y}, \hat{Z}] \begin{bmatrix} I & 0 \\ S & T \end{bmatrix} = [\hat{Y} + \hat{Z}S, \hat{Z}T],$$

and

$$Z_N^{-T} Z_B^T = \hat{Z}_N^{-T} T^{-T} T^T \hat{Z}_B^T = \hat{Z}_N^{-T} \hat{Z}_B^T = -N^T B^{-T}. \quad (3.11)$$

Since the edge directions emanating from (3.9) are the columns of the matrix $(-Z_N^{-T} Z_B^T)$ we have proved our claim. Also note that the dual reduced cost vector $b^T(Y_B^T - Y_N^T Z_N^{-T} Z_B^T)$ equals $b^T B^{-T}$ and s_N equals the vector of nonbasic primal reduced costs, since from (3.10) and (3.11),

$$Y_B^T - Y_N^T Z_N^{-T} Z_B^T = \hat{Y}_B^T + S^T \hat{Z}_B^T - (\hat{Y}_N^T + S^T \hat{Z}_N^T) \hat{Z}_N^{-T} \hat{Z}_B^T = B^{-T}$$

and

$$Z_N^{-T} Z^T = \hat{Z}_N^{-T} \hat{Z}^T = (-N^T B^{-T}, I).$$

If we now define ρ_i to be the i th row of $[I, B^{-1}N]$, i.e., $\rho_i = e_i^T B^{-1}A$, then the recurrence formulas (3.3a) and (3.3b) remain valid, as do the recurrence formulas (3.4a) and (3.4b) with the change that τ is now defined as $\tau = B^{-1}v$, where v is defined below (3.7). Also β_p can be computed explicitly as $\beta_p = \alpha^T \alpha$. Thus we see that this dual steepest-edge algorithm falls in between the previous two variants. We note that this was the first dual steepest-edge algorithm to be proposed (Goldfarb, 1976).

3.4. Upper bounds

Let us now consider the dual of problem (1.1). If we associate a nonnegative dual variable v_j with each upper bound constant $x_j \leq u_j$ ($j=1, \dots, n$) and introduce a nonnegative dual slack variable w_j ($j=1, \dots, n$) for each dual constraint (w_j is associated with the constraint $x_j \geq 0$) then this dual problem is

$$\begin{aligned} & \text{maximize} && b^T y - u^T v \\ & \text{subject to} && A^T y - v + w = c, \\ & && v \geq 0, \quad w \geq 0. \end{aligned} \quad (3.12)$$

A basic dual feasible solution (y, v, w) , where $v^T = (v_B^T, \underline{v}^T, \bar{v}^T)$ and $w^T = (w_B^T, \underline{w}^T, \bar{w}^T)$ correspond to the partition of $A = [B, \underline{N}, \bar{N}]$ into basic columns B , nonbasic columns associated with $x_j = 0$, $\underline{N}$, and nonbasic columns associated with $x_j = u_j$, $\bar{N}$, satisfies (by complementarity) the $(2n + m) \times (2n + m)$ system of equations

$$\begin{aligned} B^T y - v_B + w_B &= c_B, \\ \underline{N}^T y - \underline{v} + \underline{w} &= c, \\ \bar{N}^T y - \bar{v} + \bar{w} &= \bar{c}, \\ \underline{v} &= 0, \quad \bar{w} = 0, \quad w_B = 0, \quad v_B = 0. \end{aligned} \quad (3.13)$$

If the v and w variables are eliminated from (3.13), one obtains the system

$$\begin{aligned} B^T y &= c_B, \\ \underline{N}^T y &\leq c, \\ \bar{N}^T y &\geq \bar{c}. \end{aligned} \quad (3.14)$$

Choosing variable x_p to leave the basis on a dual simplex pivot step corresponds to relaxing the p th equality constraint in (3.14) to $a_p^T y \leq c_p$ if x_p becomes nonbasic at its lower bound of zero, and $a_p^T y \geq c_p$ if x_p becomes nonbasic at its upper bound of u_p . In the former case the edge direction in the space of dual variables y is $-\sigma$, where $B^T \sigma = e_p$ while in the latter case it is σ , and it is evident that the recurrence formulas (3.4a and 3.4b) for the edge direction norms with $B\tau = \sigma$ still apply.

In the $(2n + m)$ -dimensional space of (y, v, w) , the edge direction corresponding to x_p leaving the basis is $e_p^T [-B^{-1}, B^{-1}\underline{N}, -B^{-1}\bar{N}, 0, 0, I, 0]$ if x_p goes to zero and $e_p^T [B^{-1}, -B^{-1}\underline{N}, B^{-1}\bar{N}, 0, 0, 0, I]$ if x_p goes to u_p , where we have assumed that the variables are ordered as $(y, \underline{w}, \bar{v}, \underline{v}, \bar{w}, w_B, v_B)$. To verify this, we note that if M represents the matrix of the full $(2n + m) \times (2n + m)$ system of linear equations (3.13) with its columns ordered to correspond to the ordering of the variables specified above, then

$$M^{-1} = \begin{bmatrix} B^{-T} & & & -B^{-T} & & B^{-T} \\ -\underline{N}^T B^{-T} & I & & \underline{N}^T B^{-T} & & -\underline{N}^T B^{-T} \\ \bar{N}^T B^{-T} & & -I & \bar{N}^T B^{-T} & & \bar{N}^T B^{-T} \\ & & & I & & \\ & & & & I & \\ & & & & & I \end{bmatrix}. \quad (3.15)$$

Furthermore, when x_q leaves the basis and goes to zero, the p th component of w_B is no longer required by complementarity to be zero, while if x_q goes to u_q , the p th component of v_B is no longer required to be zero. Since $w_B = 0$ and $v_B = 0$ are the last $2m$ equations in (3.13) and the only equation that is relaxed in (3.13) is one of these, it is obvious that the corresponding column of M^{-1} (one of the last $2m$ columns) is

the desired edge direction. Again it is clear that the recurrence formulas (3.4a) and (3.4b) for the β_i with τ defined by (3.7) still apply.

Finally if the free variables y are eliminated from problem (3.12) and M represents the matrix of the resulting $2n \times 2n$ system of linear equations corresponding to the vector of variables $(w, \bar{v}, \underline{v}, \bar{w}, w_B, v_B)$, then M^{-1} is just the submatrix of the last $2n$ rows and columns of the matrix (3.15). Moreover, the comments below (3.15) apply to this case as well. Hence in all three variants of the dual steepest-edge simplex algorithm the recurrence formulas (3.4a) and (3.4b) for the edge direction norms, with τ computed appropriately for each case, hold whether or not there are upper as well as lower bounds on the primal variables.

For the linear programming problem (1.1) with upper bounds on the variables it is possible to derive two additional steepest-edge algorithms by eliminating either the v or the w variables, but not both, from formulation (3.12) of the dual (1.1). After elimination of appropriate equations and variables in (3.13) and the corresponding rows and columns of M^{-1} in (3.15), one obtains edge directions that are the rows of $(-B^{-1}, B^{-1}, \underline{N}, I)$ and $(B^{-1}, B^{-1}\bar{N}, I)$ respectively.

4. Approximate steepest-edge algorithms

Approximate steepest-edge simplex algorithms are variants of steepest-edge algorithms in which the edge direction norms used in the pivot rule are replaced by approximate weights. As already mentioned, Harris' Devex Code implements one such algorithm. This algorithm uses weights d_j to approximate the norms of the subvectors $\tilde{\eta}_j$ consisting of only those components of the edge direction vectors η_j in a so-called "reference framework". This framework is initially and periodically set to the current set of nonbasic variables. Consequently, initially and whenever the reference framework is redefined, all d_j are set equal to one. At other iterations the d_j are updated by the formulas

$$\bar{d}_p = \max\{1, \|\tilde{\eta}_q\|/\alpha_q\}, \quad (4.1a)$$

$$\bar{d}_j = \max\{d_j, |\bar{\alpha}_j| \cdot \|\tilde{\eta}_q\|\}, \quad j > m, j \neq q. \quad (4.1b)$$

Since η_q is computed explicitly in the simplex method, $\tilde{\eta}_q$ is available and $\|\tilde{\eta}_q\|$ can be easily computed. Although $\|\eta_q\| \geq 1$, $\|\tilde{\eta}_q\|$ can be very small if q is not in the current reference framework; hence (4.1a) is used to ensure that all weights d_j are greater than or equal to one. Formula (4.1b) is a consequence of using the larger of the norms of the two vectors η_j and $-\bar{\alpha}_j\eta_q$ to approximate the norm of their sum $\tilde{\eta}_j = \eta_j - \bar{\alpha}_j\eta_q$. Clearly, as long as a variable remains nonbasic, its weight d_j cannot decrease. For this reason it is necessary to reset the reference framework periodically.

Harris (1973) also briefly described a dual version of the Devex algorithm in her paper. This dual Devex algorithm uses weights h_i to approximate the norms of the subvectors $\tilde{\rho}_i$ consisting of only those components of the dual edge direction vectors ρ_i in a reference framework. This framework is initially and periodically set to the

current set of basic dual slack variables; hence the edge directions ρ_i are those of the dual polyhedron corresponding to either problem (3.5) or problem (3.8).

From an alternative point of view, the elements of the subvector $\tilde{\rho}_i$ are just the components of the i th row of the primal simplex tableau in columns that were basic when the reference framework was chosen. In the primal Devex algorithm, the elements of $\tilde{\eta}_j$ are the components of the j th column of the simplex tableau in rows corresponding to variables that were nonbasic when the reference framework was chosen.

Whenever a new reference framework is chosen, h_i is set equal to one for all i . At other iterations the h_i are updated by the formulas

$$\bar{h}_p = \max\{1, \|\tilde{\alpha}\|/|\alpha_q|\}, \quad (4.2a)$$

$$\bar{h}_p = \max\{h_i, |w_i/w_p| \cdot \|\tilde{\alpha}\|\}, \quad i \leq m, \quad i \neq p, \quad (4.2b)$$

where α is the pivot row of the tableau, which we have assumed to be the p th row. Note that $w_p = \alpha_q$. In our implementations of both the primal and dual simplex method, we explicitly compute α ; hence $\|\tilde{\alpha}\|$ is readily computed.

In the primal Devex algorithm it makes sense to consider redefining the reference framework and reinitializing the edge weights whenever the weight d_q corresponding to the entering variable differs substantially from the norm $\|\tilde{\eta}_q\|$ it is supposed to approximate. Similarly in the dual Devex algorithm, when h_p is no longer a good approximation to $\|\tilde{\alpha}\|$, the reference framework should be redefined. Resetting the reference framework is clearly important in Devex algorithms since the edge weights can only increase. Hence they may eventually become very large, impairing the effectiveness of the algorithms.

5. Projected steepest-edge algorithms

It is possible to develop steepest-edge algorithms that are based upon the exact norms of the subvectors $\tilde{\eta}_j$ in the primal case and $\tilde{\rho}_i$ in the dual case rather than approximations to these norms. Clearly, the recurrence formulas (2.4a) and (2.4b) apply to $\tilde{\eta}_j$ in place of η_j , and the recurrence formulas (3.3a) and (3.3b) apply to $\tilde{\rho}_i$ in place of ρ_i . Consequently, if we define $\gamma_j \equiv \|\tilde{\eta}_j\|^2$ and $\beta_i \equiv \|\tilde{\rho}_i\|^2$, then it is easily verified that the recurrence formulas (2.5a) and (2.5b) for γ_j and (3.4a) and (3.4b) for β_i remain valid with two exceptions. These are that w in (2.5b) is replaced by $\tilde{w}$, the subvector of w consisting of components in the reference framework, and τ in (3.4b) is replaced by $\tau = B^{-1}\tilde{v}$, where $\tilde{v} = \sum_{j=m+1}^n \tilde{\alpha}_j a_j$ and $\tilde{\alpha}$ is the subvector of the pivot row $e_p^T B^{-1}A$ consisting of components in the (dual) reference framework. We shall refer to algorithms that use these norms (actually seminorms of η_j and ρ_i) to determine the simplex pivots and the above recurrence formulas as “projected steepest-edge” algorithms. Observe that γ_q in (2.5a) and β_p in (3.4a) can be computed explicitly since $\|\tilde{\eta}_q\|^2 = \delta + \tilde{w}^T \tilde{w}$, where δ equals one if q is in the reference framework and is zero otherwise, and $\|\tilde{\rho}_p\|^2 = \tilde{\alpha}^T \tilde{\alpha}$.

Although any set of variables can be used as the reference framework, those used in the Devex algorithms — i.e., the current nonbasic variables in the primal case, and the current basic variables in the dual case — allow us to set $\|\tilde{\eta}_j\| = 1$ for all nonbasic j in the primal algorithm, and $\|\tilde{\rho}_i\| = 1$ for all i , $1 \leq i \leq m$, in the dual algorithm whenever the reference framework is redefined. Furthermore these choices ensure that the norms do not vanish on subsequent iterations.

To verify this for the dual algorithm, observe that initially $\tilde{\rho}_i = (\widetilde{e_i^T B^{-1} A}) = e_i^T I$, where B is the initial basis matrix. Clearly, at a subsequent iteration where the basis matrix is $\bar{B}$, $\tilde{\rho}_i = (\widetilde{e_i^T \bar{B}^{-1} A}) = e_i^T \bar{B}^{-1} B \neq 0$, since both B and $\bar{B}$ are nonsingular. For the primal algorithm, initially the $\tilde{\eta}_j$, $j = m+1, \dots, n$, are the last $n-m$ components of the last $n-m$ columns of

$$M^{-1} = \begin{bmatrix} B^{-1} & -B^{-1}N \\ 0 & I \end{bmatrix}, \quad (5.1)$$

which is the inverse of the matrix of active constraint normals

$$M = \begin{bmatrix} B & N \\ 0 & I \end{bmatrix}. \quad (5.2)$$

That is, $\tilde{\eta}_j$ ($j > m$) is the $(j-m)$ th column of the submatrix I in (5.1). Assume for simplicity that at a subsequent iteration variables $x_{k+1}, x_{k+2}, \dots, x_{m+k}$ are basic (i.e., the first k nonbasic variables replace variables $x_1, \dots, x_k$ in the basis). The matrix $\bar{M}$ of active constraint normals at this iteration is then equal to M in (5.2) premultiplied by

$$\begin{pmatrix} I & 0 \\ S & T \end{pmatrix}, \quad (5.3)$$

where S is a matrix whose first k rows are the first k rows of B^{-1} and whose other rows are all zero, and T is an identity matrix with its first k rows replaced by the first k rows of $-B^{-1}N$. Postmultiplying M^{-1} in (5.1) by the inverse of the transformation matrix (5.3) we obtain $\bar{M}^{-1}$ and it is easy to check that the submatrix consisting of the last $n-m$ rows and columns of $\bar{M}^{-1}$ is just T^{-1} . (The nonsingularity of T follows from the nonsingularity of (5.3).) Therefore, at this iteration the $\tilde{\eta}_j$ corresponding to nonbasic variables are the columns of T^{-1} and cannot vanish.

We now shall show that projected steepest-edge algorithms can be interpreted as “pure” steepest-edge algorithms for a particular reformulation of the underlying primal or dual linear program. First let us consider the primal algorithm and the reformulation of the primal linear program (2.1) obtained after eliminating the basic variables from the problem using the system of equation $Bx_B + Nx_N = b$; i.e.,

$$\begin{aligned} & \text{minimize} && \bar{c}_N^T x_N - \bar{c}_B^T B^{-1} b \\ & \text{subject to} && -B^{-1} N x_N \geq -B^{-1} b, \\ & && x_N \geq 0, \end{aligned} \quad (5.4)$$

where $\bar{c}_N^T = c_N^T - c_B^T B^{-1}N$, the vector of reduced costs corresponding to the basis B . If at some iteration of the projected steepest-edge algorithms variables $x_{k+1}, x_{k+2}, \dots, x_{m+k}$ are basic, then the matrix of active constraint normals in (5.4) is the matrix T defined above. Initially $k=0$ and $T=I$. But the edge directions of (5.4) corresponding to this basis are the columns of T^{-1} and we have already shown in the discussion above that these columns are the subvectors $\tilde{\eta}_j$ corresponding to the nonbasic variables. Moreover, $c^T \eta_j = (c^T - c_B^T B^{-1}A)\eta_j = \bar{c}_N^T \tilde{\eta}_j$ since $A\eta_j = 0$. This completes our proof that the projected primal steepest-edge algorithm for which the reference framework is the last $n-m$ variables corresponds to the primal steepest-edge algorithm applied to (5.4). Similarly, one can show that the projected dual steepest-edge algorithm for which the reference framework is the set of dual slack variables $s_1, s_2, \dots, s_m$ corresponds to a steepest-edge algorithm applied to the dual problem

$$\begin{aligned} &\text{maximize} && -b^T B^{-T}(s_B - c_B) \\ &\text{subject to} && N^T B^{-T} s_B \geq -\bar{c}_N, \\ &&& s_B \geq 0, \end{aligned} \tag{5.5}$$

obtained by eliminating the dual slacks s_N from (3.8). To be specific we have set $[Y, Z]$ in (3.8) equal to M^{-1} in (5.1).

If the constraint matrices in problems (5.4) and (5.5) are not scaled initially, the projected steepest-edge algorithms may not perform well if B is ill-conditioned. Scaling requires the nontrivial computation of $B^{-1}N$. Alternatively, the reference framework can be redefined if any of the norms of the $\tilde{\eta}_j$ or $\tilde{\rho}_i$ drop below some tolerance.

Instead of simply resetting the reference framework in a projected steepest-edge algorithm when progress is impaired, a better strategy is to allow this set to grow. Specifically we have in mind “dynamic” steepest-edge algorithms in which the leaving variable x_p in the primal algorithm and the entering variable x_q in the dual algorithm is added to the reference framework if they are not already in it.

If such a change is made to the projected steepest-edge algorithms, all nonbasic variables in the primal algorithm and all basic variables in the dual algorithm will always be in the reference framework. Moreover, the change is trivial to implement. In both algorithms the weights γ_j and β_i are first updated exactly as in the “static” projected algorithm. Then, in the primal algorithm if $p \notin R$ (the reference framework), we set $R \leftarrow R \cup \{p\}$ and $\gamma_p \leftarrow \gamma_p + 1$, and in the dual algorithm if $q \notin R$, we set $R \leftarrow R \cup \{q\}$ and $\beta_p \leftarrow \beta_p + 1$. If R eventually grows to the full set of indices $\{1, \dots, n\}$ then the dynamic primal algorithm becomes the primal steepest-edge algorithm, while the dynamic dual algorithm becomes the third variant of the dual steepest-edge algorithm described in Section 3.

It is also possible to allow variables to leave the reference framework. If such a variable is the variable x_q entering the basis in the primal algorithm and the variable x_p leaving the basis in the dual algorithm, the changes that need to be made in the

projected algorithms are extremely simple. If $q \in R$ is to be dropped from R in the primal algorithm, then we set $R \leftarrow R - \{q\}$ and compute $\gamma_q = \tilde{w}^T \tilde{w}$ (i.e., $\gamma_q \leftarrow \gamma_q - 1$) before updating the γ_j , $j \neq q$. If $p \in R$ is to be dropped from R in the dual algorithm, then we set $R \leftarrow R - \{p\}$ and compute $\beta_p = \tilde{\alpha}^T \tilde{\alpha}$ (i.e., $\beta_p \leftarrow \beta_p - 1$) before updating the β_i , $i \neq p$.

While it makes sense to always add to R the variable leaving the basis in the primal algorithm and the variable entering the basis in the dual algorithm, when to drop q and p respectively from R is not so clear. One possible strategy in the primal algorithm is to drop q from R only if x_q is a slack variable that goes into the basis at a reasonably large value.

6. Computational results

In this section we present computational results comparing various exact and inexact steepest-edge simplex algorithms with one another and with simplex algorithms that use the most negative reduced cost (in the dual case, the most negative basic variable) — the so-called Dantzig pivot rule. Based on experience with modern computer architectures (Forrest and Tomlin, 1990), there is no default partial pricing strategy in OSL for the primal simplex method. It was therefore decided to compare steepest-edge methods only with the Dantzig rule (but see Lustig, 1989, for another viewpoint). All algorithms were implemented as options in the SSLV collection of simplex codes, which is a part of the Optimization System Library OSL (1991), and all test results reported here were obtained on an IBM RS/6000 Model 540 workstation.

SSLV for the IBM RS/6000 is a set of FORTRAN routines that can solve linear programs by either the primal or the dual simplex method. These routines provide a robust and stable implementation of the simplex method and have been written to take advantage of the IBM RS/6000 RISC architecture, especially in pricing operations. We now briefly describe several of SSLV's important features. A more complete description can be found in the OSL User's Guide (1991) and in Forrest and Tomlin (1991).

SSLV maintains an LU factorization of the basis matrix using a modified version of the updating method of Forrest and Tomlin (1972) suggested by Saunders (1976), where the permuted spike is placed in U at the position of its last nonzero, rather than at the end of U . When refactorization is called for, a "dynamic Markowitz method" is used to select the pivot sequence so as to balance numerical stability considerations (the size of the pivot element) against sparsity considerations (the growth of nonzeros in the factorization) as described in (Suhl and Suhl, 1990).

Pivot row selection in the primal simplex method and pivot column selection in the dual simplex method is done using ideas due to Harris (1973). Harris' approach essentially treats near-degeneracy as degeneracy by allowing infeasibilities up to a prescribed tolerance so as to increase the number of possible pivot choices. Amongst

these the largest pivot element is chosen. This scheme tries to avoid very small pivots which can markedly increase the ill-conditioning of the basis matrix. It also appears to reduce the number of degenerate pivots taken. Variables are also allowed to violate their bounds slightly when they are not in the basis. Such nonbasic variables are kept fixed at their current infeasible values while they remain nonbasic.

The code uses a *composite* objective function throughout the solution process (Wolfe, 1965). This objective function is a weighted combination of the original objective function and the *sum of the infeasibilities*. The weight on the latter is fixed at one while the weight on the original objective function is initially set to balance the amount of primal and dual infeasibility and is reduced whenever progress towards reducing the sum of the infeasibilities is not up to target. Normally this weight is orders of magnitude less than one.

If in the course of several hundred iterations ninety percent of the pivots are degenerate and nonconstructive — i.e., an artificial variable is not removed from the basis — SSLV applies a small random perturbation to the problem. (An artificial variable is a slack variable whose lower and upper bounds are equal.) In the primal algorithms this perturbation changes the right hand side by an amount that affects only the current degenerate basic variables. In the dual algorithm the costs corresponding to degenerate dual slacks are perturbed.

In all runs reported below, the problems were first reduced in size by a preprocessor to remove redundant rows and eliminate or fix variables where possible. The problems were then scaled unless the ratio of the moduli of the maximum element to the minimum nonzero element in A was less than 10 and the moduli of all nonzero elements were between 0.1 and 10. The scaling routine makes several passes alternately scaling the rows and columns of the constraint matrix. Each pass makes the product of maximum and minimum moduli of nonzero elements in every column and row, respectively, equal to one. A final pass then scales the matrix so that in each column the maximum modulus of an element equals one. Also before starting simplex iterations, all algorithms performed a “crash” to obtain an initial basis. The crash routine tries to get a triangular basis by choosing “pivots” in columns with zero cost coefficients unless the pivots happen to be in GUB rows.

Before we present our computational results, some remarks about the test problems are in order. We first tested our algorithms on essentially all of the problems in the NETLIB test set that are large ($>10\,000$ nonzeros in the constraint matrix before preprocessing) and, from past experience, appear to be at least moderately difficult (required more than $2m$ iterations for solution as reported in Bixby, 1990). Several problems that meet these criteria (*pilotnov*, *fit1p*, and *fit1d*) were not run, as larger variants of these problems were included in our tests. Also, problems *truss* and *stocfor3* were not tried because of the very large storage requirements needed by the generators and data for these problems.

Of the fourteen NETLIB problems used in our tests, only *dff001* is very large in terms of the basis size (i.e., number of rows, m). Consequently, we added six large and/or difficult problems to our test set: *aamat*, a crew scheduling problem; *grade4*,

a personnel assignment problem; *air2* and *model3*, two fleet scheduling problems; *kk1* and *aggint*, two problems whose type is unknown to us.

Our computational results are reported in Tables 1–20. Each table presents the outcome of using five different primal algorithms and eight different dual algorithms on a given problem, whose characteristics are listed on the first line. The numbers preceding *rows* and *columns* are the number of rows and columns in the problem after preprocessing, while the two numbers, the second inside parentheses, preceding *nonzeros* are the numbers of nonzeros in the constraint matrix and the objective function gradient before preprocessing. The weight initially applied to the original objective in the composite objective function is given above the statistics for each set of algorithms. All times reported are in seconds and include the time for the crash but not the time required for preprocessing the problem. The time and number of iterations (pivots) needed to obtain an optimal feasible solution from the solution to the perturbed problem (when perturbations were introduced during the course of the algorithm) is included in the totals reported. The columns labeled *before* and *after* give, respectively, the number of iterations before and after a feasible solution was obtained. The column labeled % *Zero pivots* gives the percentage of pivots that were degenerate and that did not remove an artificial variable from the basis. Somewhat surprisingly the ratio of the numbers of zero pivots to total pivots was not very

Table 1

Problem 25FV47, 715 rows, 1484 columns, 9994(727) nonzeros

Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	3124	34.3	802	2322	6.7
Devex (28)	2172	25.2	687	1485	6.5
Projected	1205	17.1	525	680	7.6
Dynamic	1200	17.0	525	675	7.7
Steepest (0.80)	1164	16.9	576	588	5.2

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	3098	39.2	107	2991	14.5
Devex (136)	3096	42.6	109	2987	14.7
Projected	2038	37.2	94	1944	21.7
Dynamic	1900	36.1	111	1789	20.0
Uninit steepest	1922	31.1	63	1859	25.3
Steepest 1 (0.50)	2048	34.3	54	1994	21.9
Steepest 2 (2.83)	2164	44.6	72	2092	13.4
Steepest 3 (2.75)	1975	39.2	72	1903	23.1

Table 2

Problem PILOT.JA, 771 rows, 1535 columns, 11235(8) nonzeros

Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	5433	67.2	934	4499	1.0
Devex (20)	2063	27.6	680	1383	0.6
Projected	1709	26.2	685	1024	0.9
Dynamic	1604	25.0	695	909	0.7
Steepest (0.92)	1482	30.7	585	897	0.7

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	2432	34.4	328	2104	1.6
Devex (31)	2099	37.5	248	1851	1.2
Projected (1)	1508	30.4	281	1227	2.7
Dynamic	1456	30.8	334	1122	2.5
Uninit steepest (4)	2712	56.0	429	2283	2.1
Steepest 1 (0.52)	1616	33.1	159	1457	3.2
Steepest 2 (3.26)	2080	48.6	160	1920	2.7
Steepest 3 (3.15)	1592	38.3	164	1428	2.2

Table 3

Problem DEGEN3, 1412 rows, 1727 columns, 24464(1586) nonzeros

Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	12494	234.6	3600	8894	4.0
Devex (27)	3558	65.5	2147	1411	3.3
Projected	1779	40.2	1074	705	3.0
Dynamic	1975	44.4	1145	830	3.1
Steepest (1.53)	1893	44.2	1104	789	3.0

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	3866	89.7	17	3849	0.9
Devex (84)	2733	64.4	17	2716	2.3
Projected	1165	35.2	17	1148	3.7
Dynamic	1158	35.1	17	1141	4.4
Uninit steepest	1112	30.5	17	1095	3.6
Steepest 1 (2.81)	1224	36.2	17	1207	4.0
Steepest 2 (10.18)	1228	47.8	17	1211	4.2
Steepest 3 (9.99)	1159	46.1	17	1142	4.1

Table 4
Problem NESM, 614 rows, 2708 columns, 12990(700) nonzeros

Initial primal weight 0.10000000 D +00					
Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	3790	36.6	2275	1515	0.0
Devex (42)	3743	39.2	2488	1255	0.0
Projected	2822	29.8	1994	828	0.0
Dynamic	2880	30.8	2273	607	0.0
Steepest (1.51)	2897	34.3	2282	615	0.0
Initial dual weight 0.10000000 D -07					
Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	4006	46.1	0	4006	3.3
Devex (247)	6123	79.2	0	6123	0.8
Projected	2446	44.8	0	2446	1.3
Dynamic (1)	4306	84.2	0	4306	0.7
Uninit steepest	2472	38.0	0	2472	1.3
Steepest 1 (0.32)	2545	39.5	0	2545	1.4
Steepest 2 (3.25)	4526	94.7	0	4526	0.7
Steepest 3 (3.23)	4625	94.4	0	4625	0.7

Table 5
Problem BNL2, 1188 rows, 2506 columns, 11642(2235) nonzeros

Initial primal weight 0.10000000 D +00					
Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	4655	65.2	2050	2605	0.7
Devex (24)	2505	39.1	1473	1032	0.8
Projected	1679	28.2	1133	546	0.7
Dynamic	1504	25.2	1059	445	0.9
Steepest (2.72)	1915	35.0	1180	735	0.9
Initial dual weight 0.10000000 D -07					
Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	2111	30.6	73	2038	3.0
Devex (68)	1806	28.0	84	1722	3.2
Projected	1137	25.0	74	1063	2.9
Dynamic	1118	25.0	70	1048	3.0
Uninit steepest	1106	19.4	73	1033	2.9
Steepest 1 (0.96)	1134	20.7	64	1070	3.2
Steepest 2 (7.50)	1114	30.9	57	1057	3.1
Steepest 3 (7.24)	1106	30.4	57	1049	3.0

Table 6

Problem PILOTS, 1369 rows, 3361 columns, 40736(53) nonzeros

Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	10559	483.8	2738	7821	0.0
Devex (37)	5153	251.1	2476	2677	0.1
Projected	3348	194.9	1773	1575	0.1
Dynamic	3545	210.0	2007	1538	0.1
Steepest (6.01)	3982	236.5	2716	1266	0.2

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	13542	519.6	1484	12058	2.1
Devex (370)	10073	444.2	2284	7789	2.1
Projected (3)	4339	247.3	1258	3081	2.0
Dynamic	4234	262.2	1412	2822	1.8
Uninit steepest (2)	4315	254.7	1093	3222	1.9
Steepest 1 (2.72)	3679	212.6	953	2726	1.9
Steepest 2 (15.76)	3390	242.9	519	2871	2.1
Steepest 3 (15.49)	3602	252.0	506	3096	2.1

Table 7

Problem GREENBEB 1490 rows, 3664 columns, 23132(622) nonzeros

Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	8759	188.0	3012	5747	1.3
Devex (25)	3380	79.7	1771	1609	1.4
Projected	2551	71.7	1656	895	1.2
Dynamic	2219	61.1	1162	1057	1.4
Steepest (4.04)	2588	73.2	1677	911	1.4

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	7213	186.5	880	6333	3.4
Devex (210)	6992	229.8	1368	5624	2.9
Projected	4064	173.8	918	3146	2.2
Dynamic	4248	192.0	1032	3216	2.4
Uninit steepest (1)	4019	137.6	401	3618	0.9
Steepest 1 (3.22)	3671	129.9	413	3258	1.1
Steepest 2 (14.11)	3495	154.5	409	3086	1.0
Steepest 3 (13.91)	3055	135.1	533	2522	0.8

Table 8
Problem GREENBEA 1492 rows, 3675 columns, 23217(622) nonzeros

Initial primal weight 0.10000000 D + 00					
Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	9082	195.8	2716	6366	1.0
Devex (31)	4274	101.9	1670	2604	1.7
Projected	4038	105.5	1665	2373	1.7
Dynamic	4272	115.3	1511	2761	1.7
Steepest (3.94)	3771	105.1	1582	2189	1.7
Initial dual weight 0.10000000 D - 07					
Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	6487	221.3	2639	3848	1.8
Devex (125)	5602	206.1	3050	2552	2.1
Projected	2992	125.1	1802	1190	1.8
Dynamic	3249	150.8	2082	1167	1.7
Uninit steepest	4185	176.9	2943	1242	1.6
Steepest 1 (3.25)	3435	153.1	2323	1112	1.5
Steepest 2 (14.17)	3292	166.7	2066	1226	1.8
Steepest 3 (13.96)	3199	169.0	2103	1096	1.7

Table 9
Problem PILOT87, 1977 rows, 4610 columns, 70483(683) nonzeros

Initial primal weight 0.10000000 D + 00					
Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	19852	1968.7	2794	17058	0.0
Devex (47)	8000	738.6	3327	4673	0.1
Projected (1)	5157	560.3	2456	2701	0.1
Dynamic	4957	533.7	2342	2615	0.1
Steepest (8.48)	5815	601.0	2684	3131	0.1
Initial dual weight 0.10000000 D - 07					
Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	24757	2477.3	1729	23028	1.6
Devex (803)	20018	1810.2	2265	17753	1.3
Projected (10)	9811	1167.9	2169	7642	1.9
Dynamic (1)	8892	965.6	2184	6708	1.8
Uninit steepest (14)	13427	1579.8	1928	11499	1.7
Steepest 1 (6.30)	8119	889.1	1292	6827	1.3
Steepest 2 (35.15)	9219	1112.4	1747	7472	1.9
Steepest 3 (34.83)	10510	1316.8	2296	8214	1.8

Table 10

Problem D2Q06C, 1937 rows, 4993 columns, 31934(3257) nonzeros

Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	26006	773.7	3378	22628	0.5
Devex (53)	7480	237.6	2847	4633	0.9
Projected	4361	165.2	2466	1895	0.9
Dynamic	4380	168.1	2715	1665	0.8
Steepest (6.83)	4663	182.6	2622	2041	1.0

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	18843	634.7	764	18079	0.7
Devex (642)	14560	536.3	631	13929	0.6
Projected	6518	335.1	618	5900	0.8
Dynamic	6274	324.3	616	5658	0.9
Uninit steepest (3)	7816	360.6	509	7307	0.8
Steepest 1 (3.35)	6073	272.2	342	5731	1.0
Steepest 2 (23.63)	6044	343.7	257	5787	1.0
Steepest 3 (23.35)	5412	311.3	259	5153	1.0

Table 11

Problem AAMAT, 823 rows, 8904 columns, 72965(8904) nonzeros

Initial primal weight 0.17441891 D -03

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	100943	3100.8	21420	79523	4.1
Devex (276)	23386	787.0	23030	356	2.3
Projected	5114	249.5	1714	3400	2.0
Dynamic	5423	268.2	2082	3341	2.3
Steepest (6.46)	4781	249.2	1647	3134	2.1

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	15395	787.5	0	15395	2.2
Devex (215)	7596	391.8	0	7596	1.3
Projected	3265	210.3	0	3265	0.9
Dynamic	3141	206.2	0	3141	0.9
Uninit steepest	3482	191.7	0	3482	0.8
Steepest 1 (0.68)	3535	193.7	0	3535	0.9
Steepest 2 (18.97)	3025	250.8	0	3025	0.8
Steepest 3 (18.92)	2986	246.2	0	2986	0.8

Table 12
Problem FIT2D, 25 rows, 10500 columns, 129018(9000) nonzeros

Initial primal weight 0.90000000 D -08					
Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	19720	277.5	0	19720	0.5
Devex (261)	9199	184.6	0	9199	0.3
Projected	5761	334.9	0	5761	0.1
Dynamic	5759	335.8	0	5759	0.1
Steepest (0.84)	5703	334.5	0	5703	0.1
Initial dual weight 0.10000000 D -07					
Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	10089	434.6	3000	7089	1.3
Devex (235)	8755	364.6	3000	5755	0.2
Projected	7721	348.4	3000	4721	0.9
Dynamic	6439	281.7	3000	3439	0.2
Uninit steepest (1)	7496	287.2	3000	4496	0.6
Steepest 1 (0.00)	8038	317.3	3000	5038	0.8
Steepest 2 (0.77)	6510	369.8	3000	3510	0.3
Steepest 3 (0.78)	6607	377.8	3000	3607	0.4

Table 13
Problem 80BAU3B, 2049 rows, 9248 columns, 20704(8061) nonzeros

Initial primal weight 0.51250955 D -01					
Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	8179	170.4	1165	7014	0.5
Devex (23)	6595	157.4	1473	5122	0.7
Projected	4562	108.8	1209	3353	0.8
Dynamic	4420	104.6	1217	3203	0.8
Steepest (17.15)	4634	123.2	763	3871	0.9
Initial dual weight 0.10000000 D -07					
Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	8962	264.8	2203	6759	1.7
Devex (250)	7831	242.4	1872	5959	1.7
Projected	6108	264.6	1782	4326	1.6
Dynamic	6091	270.0	1819	4272	1.7
Uninit steepest	5950	201.0	1800	4150	1.6
Steepest 1 (2.56)	6052	207.2	1819	4233	1.6
Steepest 2 (29.79)	6086	282.7	1711	4375	1.6
Steepest 3 (29.35)	6122	282.3	1727	4395	1.6

Table 14

Problem AIR2, 5654 rows, 8730 columns, 271082(8238) nonzeros

Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	1345993	195207.0	195552	1150441	3.6
Devex (385)	61234	8943.8	24818	36416	1.3
Projected	23976	4215.7	11342	12634	2.7
Dynamic	25222	4402.0	11681	13541	2.8
Steepest (32.99)	24158	4294.9	11777	12381	2.8

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	77463	11620.3	748	76715	0.7
Devex (795)	35900	5445.8	115	35785	0.6
Projected	9580	1815.0	96	9484	1.0
Dynamic	8565	1674.9	102	8463	1.0
Uninit steepest	9984	1756.0	101	9883	1.1
Steepest 1 (63.78)	10630	1917.4	71	10559	1.1
Steepest 2 (343.94)	8826	2302.0	68	8758	1.0
Steepest 3 (340.95)	8667	2260.6	68	8599	1.4

Table 15

Problem GRADE4, 4538 rows, 10565 columns, 148378(5904) nonzeros

Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	67836	5781.1	8974	58862	0.2
Devex (161)	23514	2194.4	6799	16715	0.2
Projected	9257	1011.8	5063	4194	0.1
Dynamic	10998	1219.6	5591	5407	0.1
Steepest (35.99)	11495	1319.3	6210	5285	0.1

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	140062	11525.6	0	140062	0.6
Devex (3168)	81058	7169.2	0	81058	0.7
Projected (5)	21689	2753.6	0	21689	1.5
Dynamic (2)	18421	2392.0	0	18421	1.6
Uninit steepest (8)	20385	2221.8	0	20385	1.5
Steepest 1 (23.26)	2856	2560.3	0	22856	1.3
Steepest 2 (171.31)	21600	3225.4	0	21600	1.5
Steepest 3 (168.82)	23202	3476.4	0	23202	1.2

Table 16
Problem DFL001, 4840 rows, 10999 columns, 33146(6241) nonzeros
Initial primal weight 0.29965557 D -02

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	1275427	93759.5	28437	1246990	5.1
Devex (332)	43204	3148.6	7527	35677	1.3
Projected	17687	1510.1	5556	12131	2.4
Dynamic	16082	1356.2	5667	10415	1.8
Steepest (40.69)	16249	1403.4	5500	10749	1.9

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	1514390	136319.2	2	1514388	1.3
Devex (265)	28699	2852.4	2	28697	1.7
Projected	17187	2173.2	2	17185	1.9
Dynamic	15828	1845.5	2	15826	1.4
Uninit steepest (5)	20634	2051.2	2	20632	1.9
Steepest 1 (23.24)	18325	1817.0	2	18323	1.5
Steepest 2 (108.06)	17685	2357.0	2	17683	1.6
Steepest 3 (106.02)	17745	2370.3	2	17743	1.8

Table 17
Problem FIT2P, 3000 rows, 13525 columns, 50284(10500) nonzeros
Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	88952	5725.5	83682	5270	0.2
Devex (51)	12884	670.8	10278	2606	0.1
Projected	7236	296.6	6883	353	0.1
Dynamic	7199	294.5	7057	142	0.0
Steepest (24.07)	9702	516.4	8714	988	0.1

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	18602	1086.4	0	18602	0.2
Devex (174)	10938	699.8	0	10938	0.1
Projected	7435	649.6	0	7435	0.1
Dynamic	7444	664.7	0	7444	0.1
Uninit steepest	8624	650.6	0	8624	0.1
Steepest 1 (6.47)	7492	555.3	0	7492	0.1
Steepest 2 (69.61)	7605	731.6	0	7605	0.1
Steepest 3 (68.72)	7764	746.7	0	7764	0.1

Table 18

Problem MODEL3, 6916 rows, 16785 columns, 54725(12730) nonzeros

Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	676121	76412.1	184698	491423	3.1
Devex (287)	48093	4941.9	8903	39190	1.3
Projected	20507	2383.2	6788	13719	1.7
Dynamic	21165	2447.8	6872	14293	1.8
Steepest (84.03)	21327	2591.2	7039	14288	1.8

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	3542756	443753.8	0	3542756	0.0
Devex (29393)	1341517	172223.4	0	1341517	0.0
Projected	23041	3674.4	0	23041	0.5
Dynamic	19491	3109.0	0	19491	0.7
Uninit steepest (2)	19241	2589.0	0	19241	0.5
Steepest 1 (51.46)	22557	3146.7	0	22557	0.5
Steepest 2 (231.10)	19518	3403.6	0	19518	0.5
Steepest 3 (227.52)	15715	2741.5	0	15715	0.5

Table 19

Problem AGGINT, 16783 rows, 54223 columns, 1567840(72126) nonzeros

Initial primal weight 0.90000000 D -08

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	34492	10630.1	0	34492	2.5
Devex (43)	8591	2582.0	0	8591	4.5
Projected	4765	1566.9	0	4765	3.9
Dynamic	5535	1833.6	0	5535	4.3
Steepest (614.93)	8607	3576.6	0	8607	5.2

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	77508	26140.3	1231	76277	6.5
Devex (1910)	119524	47940.6	14398	105126	4.5
Projected (1)	20789	10774.2	3525	17264	4.2
Dynamic (1)	39419	23213.9	8041	31378	4.6
Uninit steepest (8)	29955	13000.1	3525	26430	3.7
Steepest 1 (148.15)	20805	9071.7	3525	17280	4.1
Steepest 2 (3667.24)	35282	27307.6	6855	28427	4.3
Steepest 3 (3641.13)	33591	25703.7	7183	26408	4.0

Table 20

Problem KK1, 10662 rows, 76090 columns, 436315(67901) nonzeros

Initial primal weight 0.10000000 D +00

Primal	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	9330	1577.3	4824	4506	0.1
Devex (84)	11433	2713.8	2544	8889	0.2
Projected (9)	6456	1101.2	4435	2021	0.2
Dynamic (5)	6887	1200.6	4786	2101	0.2
Steepest (829.56)	6061	1845.0	4205	1856	0.2

Initial dual weight 0.10000000 D -07

Dual	Total		Before iters	After iters	% Zero pivots
	Iters	Time			
Dantzig	9694	3809.4	5342	4352	1.8
Devex (122)	5594	1946.2	1703	3891	1.1
Projected	3709	1594.4	962	2747	1.0
Dynamic	3807	1705.8	1104	2703	0.9
Uninit steepest	3917	1370.8	1437	2480	0.8
Steepest 1 (165.73)	3295	1298.7	1126	2169	0.7
Steepest 2 (1725.40)	3213	3258.9	1028	2185	0.8
Steepest 3 (1711.51)	3356	3309.7	1008	2348	0.8

different from one algorithm to another. Thus the reduced numbers of zero pivots taken by the steepest-edge algorithms relative to the Dantzig and Devex algorithms were a consequence rather than a cause of the reduced numbers of iterations taken by the former relative to the latter algorithms.

The dual algorithms *Steepest 1*, *2* and *3* refer to the three dual steepest-edge simplex algorithms in the order in which they are described in Section 3. The numbers appearing inside parentheses after the names of these algorithms and the primal steepest-edge algorithm are the number of seconds included in the total run times that it took to compute the initial edge weights. The primal steepest-edge algorithm spent on the average 7% of the total run time initializing the edge weights, while the dual algorithms, *Steepest 1*, *2* and *3* spent on the average, respectively, 2, 11 and 11% of the total time doing this. Clearly the initialization time for algorithm *Steepest 1* is always smaller than it is for algorithms *Steepest 2* and *3*. On the test set it was also smaller than the initialization time required for the primal algorithm *Steepest* on all problems except *degen3* and *air2*. The latter result is not surprising since $n - m$ is much smaller than m in these two problems. In fact the percentage of total run time taken by algorithm *Steepest 1* to initialize the edge weights was substantial only for problem *kk1*, on which it was 12.8%. But that was because Problem *kk1* was exceptionally easy to solve relative to its size. Algorithms *Steepest 2* and *3* spent more than half of their time, and algorithm *Steepest* nearly half of its time, initializing edge weights on this problem.

The algorithm referred to as *Uninit steepest* is an approximate steepest-edge algorithm which uses the recurrence formula used by algorithm *Steepest 1* to update approximate weights β_i , $i + 1, \dots, m$. These weights are initially set equal to one and after being updated by the recurrence formula each β_i is set equal to $\max\{\beta_i, 10^{-4}\}$. Notice that the latter computation is necessary, since otherwise the weights can become negative. We note that the primal version of this approach was suggested by Goldfarb (1976) for use with partial pricing. Algorithm *Uninit steepest* was the fastest dual algorithm on several problems (25fv47, bnl2, 80bau3b, degen3) but, not surprisingly, its performance was somewhat erratic. On the average, it was no better than the dual *projected* steepest-edge algorithm (see the summary statistics in Table 23) which also has edge weight initialization cost, and was inferior to algorithm *Steepest 1*.

The numbers appearing inside parenthesis after *Devex* and on some problems after *Projected*, *Dynamic*, and *Uninit steepest* in Tables 1–20 are the numbers of times that the reference framework was reset by these algorithms during the run. In the *Devex* algorithms this framework was reset whenever the approximate *Devex* edge weight corresponding to the entering (leaving, in the dual) variable and the exact norm of the sub-vector that this weight is supposed to approximate differed by more than a factor of three. In the *Projected*, *Dynamic* and *Uninit steepest* algorithms the reference framework was reset if, because of the build-up of roundoff errors, the freshly computed edge weight for the entering column in the primal (leaving row in the dual) differed by more than ten percent from the recurred value. As can be seen from the tables this situation rarely occurred. To try to correct under-estimates of edge weights caused by accumulated roundoff, γ_j was set to the greater of its recurred value and $1 + \bar{\alpha}_j^2$ in algorithm *Steepest*, and β_i was set to the greater of its recurred value and $1 + (w_i/w_p)^2$ in the dual algorithms *Dynamic*, *Steepest 2* and *Steepest 3*.

It can be seen from Tables 1–20 that the Dantzig pivot rule does relatively better before feasibility is obtained than after. To understand why this is so, consider the primal Dantzig pivot rule applied to a pure phase I problem. For this case the reduced cost of the entering column is $\bar{c}_q = \sum_{\bar{b}_i < 0} w_i$, where $w = B^{-1}a_q$ and $\bar{b} = B^{-1}b$. Since the Dantzig rule chooses the column with the most negative reduced cost, it favors columns for which the norm of the subvector $\{w_i | w_i < 0, \bar{b}_i < 0\}$ tends to be large. Geometrically this means that the direction chosen by the Dantzig rule tends to have a large positive component in the subspace of the basic variables that are currently negative. Hence, if a reasonable step size θ is taken, since $x_i = \bar{b}_i - \theta w_i$ for all basic variables x_i , one is likely to obtain a good improvement in the phase I (sum of infeasibilities) objective function.

The computational results reported in Tables 1–20 are summarized in Tables 21–23. Table 21 reports the ratio of the number of iterations required by each of the 13 algorithms to the sum of the number of rows and columns for each of the 20 problems. The bottom three rows give, respectively, for each of the algorithms the average of these ratios, the total number of iterations required by the algorithms

Table 21

Ratio of iterations to rows+columns

Problem	Rows	Cols	Sum	Primal				Dual								
				Dantzig	Devex	Projected	Dynamic	Steepest	Dantzig	Devex	Projected	Dynamic	Uninit	Steepest 1	Steepest 2	Steepest 3
25FV47	715	1484	2199	1.42	0.99	0.55	0.55	0.53	1.41	1.41	0.93	0.86	0.87	0.93	0.98	0.90
PILOT.JA	771	1535	2306	2.36	0.89	0.74	0.70	0.64	1.05	0.91	0.65	0.63	1.18	0.70	0.90	0.69
DEGEN3	1412	1727	3139	3.98	1.13	0.57	0.63	0.60	1.23	0.87	0.37	0.37	0.35	0.39	0.39	0.37
NESM	614	2708	3322	1.14	1.13	0.85	0.87	0.87	1.21	1.84	0.74	1.30	0.74	0.77	1.36	1.39
BNL2	1188	2506	3694	1.26	0.68	0.45	0.41	0.52	0.57	0.49	0.31	0.30	0.30	0.31	0.30	0.30
PILOTS	1369	3361	4730	2.23	1.09	0.71	0.75	0.84	2.86	2.13	0.92	0.90	0.91	0.78	0.72	0.76
GREENBEB	1490	3664	5154	1.70	0.66	0.49	0.43	0.50	1.40	1.36	0.79	0.82	0.78	0.71	0.68	0.59
GREENBEA	1492	3675	5167	1.76	0.83	0.78	0.83	0.73	1.26	1.08	0.58	0.63	0.81	0.66	0.64	0.62
PILOT87	1977	4610	6587	3.01	1.21	0.78	0.75	0.88	3.76	3.04	1.49	1.35	2.04	1.23	1.40	1.60
D2Q06C	1937	4993	6930	3.75	1.08	0.63	0.63	0.67	2.72	2.10	0.94	0.91	1.13	0.88	0.87	0.78
AAMAT	823	8904	9727	10.38	2.40	0.53	0.56	0.49	1.58	0.78	0.34	0.32	0.36	0.36	0.31	0.31
FIT2D	25	10500	10525	1.87	0.87	0.55	0.55	0.54	0.96	0.83	0.73	0.61	0.71	0.76	0.62	0.63
80BAU3B	2049	9248	11297	0.72	0.58	0.40	0.39	0.41	0.79	0.69	0.54	0.54	0.53	0.54	0.54	0.54
AIR2	5654	8730	14384	93.58	4.26	1.67	1.75	1.68	5.39	2.50	0.67	0.60	0.69	0.74	0.61	0.60
GRADE4	4538	10565	15103	4.49	1.56	0.61	0.73	0.76	9.27	5.37	1.44	1.22	1.35	1.51	1.43	1.54
DFL001	4840	10999	15839	80.52	2.73	1.12	1.02	1.03	95.61	1.81	1.09	1.00	1.30	1.16	1.12	1.12
FIT2P	3000	13525	16525	5.38	0.78	0.44	0.44	0.59	1.13	0.66	0.45	0.45	0.52	0.45	0.46	0.47
MODEL3	6916	16785	23701	28.53	2.03	0.87	0.89	0.90	149.48	56.60	0.97	0.82	0.81	0.95	0.82	0.66
AGGINT	16783	54223	71006	0.49	0.12	0.07	0.08	0.12	1.09	1.68	0.29	0.56	0.42	0.29	0.50	0.47
KK1	10662	76090	86752	0.11	0.13	0.07	0.08	0.07	0.11	0.06	0.04	0.04	0.05	0.04	0.04	0.04
Over all problems																
Average				12.43	1.26	0.64	0.65	0.67	14.14	4.31	0.71	0.71	0.79	0.71	0.73	0.72
Total rows + cols			318087													
Total iterations (1000's)				3731	290	134	137	143	5501	1721	157	165	173	157	166	162
Ratio of totals				11.73	0.91	0.42	0.43	0.45	17.29	5.41	0.49	0.52	0.54	0.49	0.52	0.51

Table 22

Ratio of iterations to primal steepest edge iterations

Problem	Rows	Cols	Sum	Primal			Dual										
				Dantzig	Devx	Projected	Dynamic	Steepest	Dantzig	Devx	Projected	Dynamic	Uninit	Steepest	1 Steepest	2 Steepest	3 Steepest
23FV47	715	1484	2199	2.68	1.87	1.04	1.03	1.00	2.66	2.66	1.75	1.63	1.65	1.76	1.86	1.70	
PILOTJA	771	1535	2306	3.67	1.39	1.15	1.08	1.00	1.64	1.42	1.02	0.98	1.83	1.09	1.40	1.07	
DEGEN3	1412	1727	3139	6.60	1.88	0.94	1.04	1.00	2.04	1.44	0.62	0.61	0.59	0.65	0.65	0.61	
NESM	614	2708	3322	1.31	1.29	0.97	0.99	1.00	1.38	2.11	0.84	1.49	0.85	0.88	1.56	1.60	
BNL2	1188	2506	3694	2.43	1.31	0.88	0.79	1.00	1.10	0.94	0.59	0.58	0.58	0.59	0.58	0.58	
PILOTS	1369	3361	4730	2.65	1.29	0.84	0.89	1.00	3.40	2.53	1.09	1.06	1.08	0.92	0.85	0.90	
GREENBEB	1490	3664	5154	3.38	1.31	0.99	0.86	1.00	2.79	2.70	1.57	1.64	1.55	1.42	1.35	1.18	
GREENBEA	1492	3675	5167	2.41	1.13	1.07	1.13	1.00	1.72	1.49	0.79	0.86	1.11	0.91	0.87	0.85	
PILOT87	1977	4610	6587	3.41	1.38	0.89	0.85	1.00	4.26	3.44	1.69	1.53	2.31	1.40	1.59	1.81	
D2Q06C	1937	4993	6930	5.58	1.60	0.94	0.94	1.00	4.04	3.12	1.40	1.35	1.68	1.30	1.30	1.16	
AAMAT	823	8904	9727	21.11	4.89	1.07	1.13	1.00	3.22	1.59	0.68	0.66	0.73	0.74	0.63	0.62	
FIT2D	25	10500	10525	3.46	1.61	1.01	1.01	1.00	1.77	1.54	1.35	1.13	1.31	1.41	1.14	1.16	
80BAU3B	2049	9248	11297	1.76	1.42	0.98	0.95	1.00	1.93	1.69	1.32	1.31	1.28	1.31	1.31	1.32	
AIR2	5654	8730	14384	55.72	2.53	0.99	1.04	1.00	3.21	1.49	0.40	0.35	0.41	0.44	0.37	0.36	
GRADE4	4538	10565	15103	5.90	2.05	0.81	0.96	1.00	12.18	7.05	1.89	1.60	1.77	1.99	1.88	2.02	
DFL001	4840	10999	15839	78.49	2.66	1.09	0.99	1.00	93.20	1.77	1.06	0.97	1.27	1.13	1.09	1.09	
FIT2P	3000	13525	16525	9.17	1.33	0.75	0.74	1.00	1.92	1.13	0.77	0.77	0.89	0.77	0.78	0.80	
MODEL3	6916	16785	23701	31.70	2.26	0.96	0.99	1.00	166.12	62.90	1.08	0.91	0.90	1.06	0.92	0.74	
AGGINT	16783	54223	71006	4.01	1.00	0.55	0.64	1.00	9.01	13.89	2.42	4.58	3.48	2.42	4.10	3.90	
KK1	10662	76090	86752	1.54	1.89	1.07	1.14	1.00	1.60	0.92	0.61	0.63	0.65	0.54	0.53	0.55	
Over all problems																	
Average				12.35	1.80	0.95	0.96	1.00	15.96	5.79	1.15	1.23	1.30	1.14	1.24	1.20	
Steepest iterations			142887														
Total iterations (1000's)				3731	290	134	137	143	5501	1721	157	165	173	157	166	162	
Ratio of totals				26.11	2.03	0.94	0.96	1.00	38.50	12.04	1.10	1.16	1.21	1.10	1.16	1.13	

Table 23

Ratio of time to primal steepest edge time

Problem	Rows	Cols	Sum	Primal				Dual								
				Dantzig	Devex	Projected	Dynamic	Steepest	Dantzig	Devex	Projected	Dynamic	Uninit	Steepest 1	Steepest 2	Steepest 3
25FV47	715	1484	2199	2.03	1.49	1.01	1.01	1.00	2.32	2.52	2.20	2.14	1.84	2.03	2.64	2.32
PILOTJA	771	1535	2306	2.19	0.90	0.85	0.81	1.00	1.12	1.22	0.99	1.00	1.82	1.08	1.58	1.25
DEGEN3	1412	1727	3139	5.31	1.48	0.91	1.00	1.00	2.03	1.46	0.80	0.79	0.69	0.82	1.08	1.04
NESM	614	2708	3322	1.07	1.14	0.87	0.90	1.00	1.34	2.31	1.31	2.45	1.11	1.15	2.76	2.75
BNL2	1188	2506	3694	1.86	1.12	0.81	0.72	1.00	0.87	0.80	0.71	0.71	0.55	0.59	0.88	0.87
PILOTS	1369	3361	4730	2.05	1.06	0.82	0.89	1.00	2.20	1.88	1.05	1.11	1.08	0.90	1.03	1.07
GREENBEB	1490	3664	5154	2.57	1.09	0.98	0.83	1.00	2.55	3.14	2.37	2.62	1.88	1.77	2.11	1.85
GREENBEA	1492	3675	5167	1.86	0.97	1.00	1.10	1.00	2.11	1.96	1.19	1.43	1.68	1.46	1.59	1.61
PILOT87	1977	4610	6587	3.28	1.23	0.93	0.89	1.00	4.12	3.01	1.94	1.61	2.63	1.48	1.85	2.19
D2Q06C	1937	4993	6930	4.24	1.30	0.90	0.92	1.00	3.48	2.94	1.84	1.78	1.97	1.49	1.88	1.70
AAMAT	823	8904	9727	12.44	3.16	1.00	1.08	1.00	3.16	1.57	0.84	0.83	0.77	0.78	1.01	0.99
FIT2D	25	10500	10525	0.81	0.54	0.97	0.97	1.00	1.26	1.06	1.01	0.82	0.83	0.92	1.07	1.10
80BAU3B	2049	9248	11297	1.38	1.28	0.88	0.85	1.00	2.15	1.97	2.15	2.19	1.63	1.68	2.29	2.29
AIR2	5654	8730	14384	45.45	2.08	0.98	1.02	1.00	2.71	1.27	0.42	0.39	0.41	0.45	0.54	0.53
GRADE4	4538	10565	15103	4.38	1.66	0.77	0.92	1.00	8.74	5.43	2.09	1.81	1.68	1.94	2.44	2.64
DFL001	4840	10999	15839	66.81	2.24	1.08	0.97	1.00	97.13	2.03	1.55	1.32	1.46	1.29	1.68	1.69
FIT2P	3000	13525	16525	11.09	1.30	0.57	0.57	1.00	2.10	1.36	1.26	1.29	1.26	1.08	1.42	1.45
MODEL3	6916	16785	23701	29.49	1.91	0.92	0.94	1.00	171.25	66.46	1.42	1.20	1.00	1.21	1.31	1.06
AGGINT	16783	54223	71006	2.97	0.72	0.44	0.51	1.00	7.31	13.40	3.01	6.49	3.63	2.54	7.64	7.19
KK1	10662	76090	86752	0.85	1.47	0.60	0.65	1.00	2.06	1.05	0.86	0.92	0.74	0.70	1.77	1.79
Over all problems																
Average				10.11	1.41	0.87	0.88	1.00	16.00	5.84	1.45	1.65	1.43	1.27	1.93	1.87
Steepest hours			4.895													
Total hours				110.19	7.76	3.89	4.08	4.90	177.78	67.43	7.36	10.41	7.50	6.36	12.72	12.21
Ratio of totals				22.51	1.58	0.80	0.83	1.00	36.32	13.77	1.50	2.13	1.53	1.30	2.60	2.49

over all problems, and the ratio of the latter to the total number of rows and columns in all problems.

On average, the three primal steepest-edge algorithms required less than $0.67(n+m)$ iterations to solve the problems in the test set, and took less than $n+m$ iterations on all but two problems (*air 2*, $1.75(n+m)$ and *df001*, $1.12(n+m)$). On average, the five dual steepest-edge algorithms required less than $0.73(n+m)$ iterations to solve these problems, and the two best dual algorithms in terms of iteration counts *Projects* and *Steepest 1* took less than $n+m$ iterations on all but three problems (*pilot87*, $1.49(n+m)$; *grade4*, $1.51(n+m)$; and *df001*, $1.16(n+m)$).

To compare the algorithms against one another we present plots of the ratios reported in Table 21 in Figures 1 and 2 for the primal and dual algorithms, respectively, and present iteration counts normalized to the iteration counts for the primal steepest-edge algorithm in Table 22. From the tables and figures we see that the steepest-edge algorithms dominate both their *Dantzig* and *Devex* counterparts on all problems, and between the latter two rules *Devex* is superior to *Dantzig* on all problems except *kk1* in the primal case and *aggint* in the dual case. These two problems, although quite large in size, are obviously easy problems to solve; hence the relative performance of the various pivot rules on them is not crucial. On average, the primal *Dantzig* and *Devex* rules took roughly 13 and 2 times as many iterations, respectively, as the best primal steepest-edge algorithm, algorithm *Projected*, and the dual *Dantzig* and *Devex* rules took roughly 14 and 5 times as many iterations, respectively as the best dual steepest-edge algorithm, algorithm *Steepest 1*. In Figure 3 we compare the best primal and dual steepest-edge algorithms, the primal *Projected* and the dual *Steepest 1* algorithms against one another. The primal algorithm takes

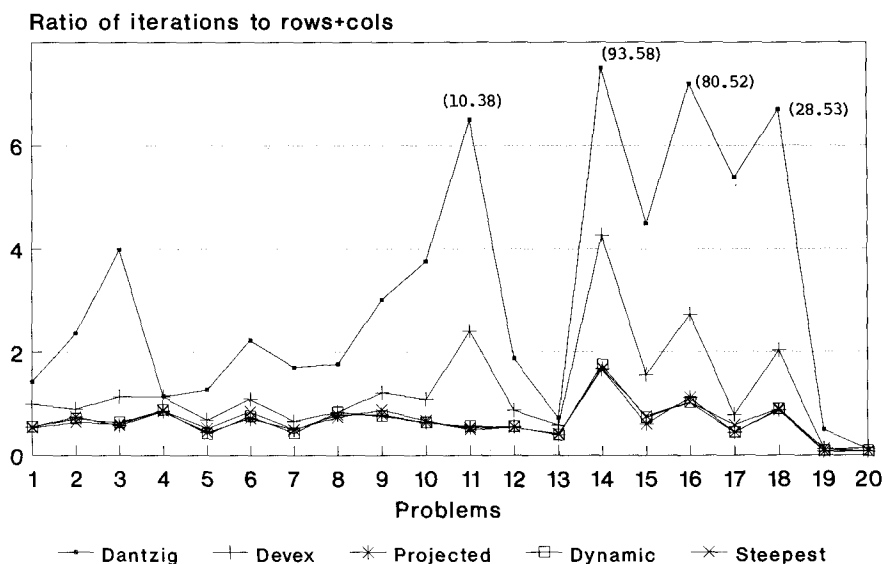


Fig. 1. Comparison of iteration counts for primal methods.

Ratio of iterations to rows+cols

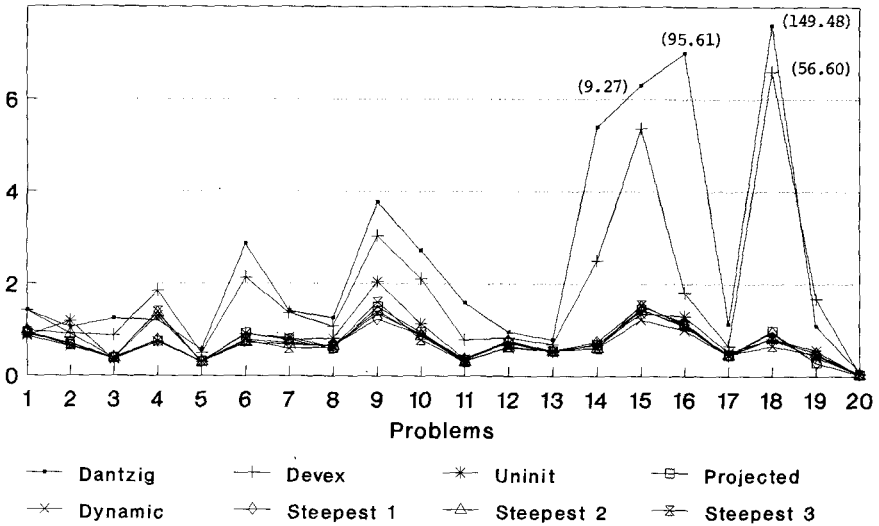


Fig. 2. Comparison of iteration counts for dual methods.

fewer iterations on 13 of the 20 problems, and we see that, in general, if a problem is relatively more difficult for one of the algorithms it is also relatively more difficult for the other algorithm. If for each problem one selects the best algorithm to solve that problem, then in our test set exactly half of the problems were solved in the

Ratio of iterations to rows+cols

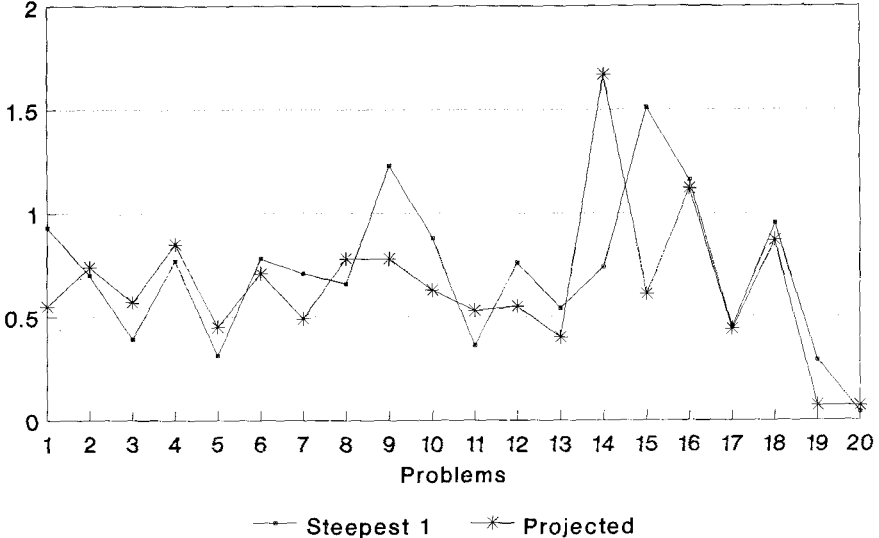


Fig. 3. Comparison between primal *Projected* and dual *Steepest 1* steepest edge methods.

fewest iterations by a primal steepest-edge algorithm and half by a dual steepest-edge algorithm.

Table 23 presents the times taken by the various algorithms to solve each problem normalized by the time taken by the primal steepest-edge algorithm. From the summary and average statistics given at the bottom of this table, we see that for our test set the primal *Projected* algorithm was the fastest primal algorithm and the dual *Steepest 1* algorithm was the fastest dual algorithm. These algorithms took 3.89 and 6.36 hours, respectively, to solve all 20 problems, which when compared with the times taken by the primal and dual *Dantzig* algorithms of 110.19 and 177.78 hours is quite impressive.

We should caution the reader that comparisons of the times taken by the dual algorithms against the times taken by the primal algorithms is not quite fair, since the code for the primal algorithms was optimized, but not much effort was made to do the same thing for the dual algorithms. For example the two B^{-T} operations required at each iteration of the primal steepest-edge algorithms were performed in the same loop of code while this was not done for the two B^{-1} operations in the dual algorithms. Also pricing and edge weight updating in the primal code was geared to the RISC/6000 architecture, but not in the dual code. Consequently, the times for the dual steepest-edge algorithms can probably be reduced by at least 10%. The dual *Dantzig* and *Devex* times can also be reduced, but not by as much.

The superiority of the primal projected steepest-edge algorithm over the primal steepest-edge algorithm is greater in terms of time than it is in terms of iteration counts, primarily because of the time taken by the latter algorithm to initialize the edge weights.

Amongst the dual steepest-edge algorithms, the time taken per iteration varies considerably. In algorithms *Steepest 2* and *3*, $v = \sum_j a_j a_j$ is computed on each iteration. Because of this and their high initialization costs, these variants are not competitive with *Steepest 1*. The dual *Projected* and *Dynamic* algorithms require the computation of $\sum_j \tilde{a}_j a_j$ on each iteration. Therefore, even though they have no initialization costs, they are not as fast as algorithm *Steepest 1*, which has a very modest initialization cost.

In summary, the steepest-edge variants of the simplex method are clearly superior to *Devex* variants and the *Dantzig* pivot rule on modern computer architectures for solving difficult large-scale linear programming problems.

References

- R.E. Bixby, "Implementing the simplex method: The initial basis," Technical Report TR90-32, Department of Mathematical Sciences, Rice University (Houston, TX, 1990).
- J.J.H. Forrest and J.A. Tomlin, "Updating triangular factors of the basis matrix to maintain sparsity in the product form simplex method," *Mathematical Programming* 2 (1972) 263–278.
- J.J.H. Forrest and J.A. Tomlin, "Vector processing in simplex and interior methods for linear programming," *Annals of Operations Research* 22 (1990) 71–100.

- J.J.H. Forrest and J.A. Tomlin, "Implementing the simplex method for the optimization subroutine library," *IBM Systems Journal* 31 (1992) 11–25.
- D.M. Gay, "Electronic mail distribution of linear programming test problems," *Mathematical Programming Society COAL Newsletter* (1985).
- D. Goldfarb, "Using the steepest-edge simplex algorithm to solve sparse linear programs," in: J. Bunch and D. Rose, eds., *Sparse Matrix Computations* (Academic Press, New York, 1976) pp. 227–240.
- D. Goldfarb and J.K. Reid, "A practical steepest-edge simplex algorithm," *Mathematical Programming* 12 (1977) 361–371.
- H. Greenberg and J.E. Kalan, "An exact update for Harris' tread," *Mathematical Programming Study* 4 (1975) 26–29.
- P.M.J. Harris, "Pivot selection methods of the Devex LP code," *Mathematical Programming* 5 (1973) 1–28.
- IBM Corp., "Optimizing subroutine library guide and reference," Document Number SC23-0519-2 (1991).
- H.W. Kuhn and R.E. Quandt, "An experimental study of the simplex method," in: Metropolis et al., eds., *Proceedings of Symposia in Applied Maths, Vol. XV* (Amer. Math. Soc., Providence, RI, 1963).
- I.J. Lustig, "An analysis of an available set of linear programming test problems," *Computers and Operations Research* 16 (1989) 173–184.
- M.A. Saunders, "The complexity of LU updating in the simplex method," in: R.S. Anderssen and R.P. Brent, eds., *The Complexity of Computational Problem Solving* (Queensland University Press, Queensland, 1976) pp. 214–230.
- U.H. Suhl and L.M. Suhl, "Computing sparse LU-factorizations for large-scale linear programming," *ORSA Journal of Computing* 2 (1990) 325–335.
- P. Wolfe, "The composite simplex algorithm," *SIAM Review* 7 (1965).
- P. Wolfe and L. Cutler, "Experiments in linear programming," in: Graves and Wolfe, eds., *Recent Advances in Mathematical Programming* (McGraw-Hill, New York, 1963).